TransLibEval: Demystify Large Language Models' Capability in Third-Party Library-Targeted Code Translation

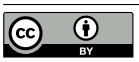
PENGYU XUE*, Shandong University, China
KUNWU ZHENG*, Shandong University, China
ZHEN YANG†, Shandong University, China
YIFEI PEI, Shandong University, China
LINHAO WU, Shandong University, China
JIAHUI DONG, Shandong University, China
XIAPU LUO, Hong Kong Polytechnic University, China
YAN XIAO, Sun Yat-sen University, China
FEI LIU, Shandong University, China
YUXUAN ZHANG, Shandong University, China
XIRAN LYU, Shandong University, China
XIANHANG LI, Shandong University, China
XUANYU ZHU, Shandong University, China
CHENGYI WANG, Shandong University, China

In recent years, Large Language Models (LLMs) have been widely studied in the code translation field on the method, class, and even repository levels. However, most of these benchmarks are limited in terms of Third-Party Library (TPL) categories and scales, making TPL-related errors hard to expose and hindering the development of targeted solutions. Considering the high dependence (over 90%) on TPLs in practical programming, demystifying and analyzing LLMs' code translation performance involving various TPLs becomes imperative.

To address this gap, we construct *TransLibEval*, the first benchmark dedicated to library-centric code translation. It consists of 200 real-world tasks across Python, Java, and C++, each explicitly involving TPLs from diverse categories such as data processing, machine learning, and web development, with comprehensive dependency coverage and high-coverage test suites. We evaluate seven recent LLMs of commercial, general, and code-specialized families under six translation strategies of three categories: Direct, IR-guided, and Retrieval-augmented. Experimental results show a dramatic performance drop compared with library-free settings (average CA decline over 60%), while diverse strategies demonstrate heterogeneous advantages. Furthermore,

*These authors contributed equally to this work.
†Corresponding author.

Authors' Contact Information: Pengyu Xue, Shandong University, Qingdao, China, xuepengyu@mail.sdu.edu.cn; Kunwu Zheng, Shandong University, Qingdao, China, xiaozhengsdu2022@mail.sdu.edu.cn; Zhen Yang, Shandong University, Qingdao, China, zhenyang@sdu.edu.cn; Yifei Pei, Shandong University, Qingdao, China, peiyifei@mail.sdu.edu.cn; Linhao Wu, Shandong University, Qingdao, China, wulinhao@mail.sdu.edu.cn; Jiahui Dong, Shandong University, Qingdao, China, 202400130128@mail.sdu.edu.cn; Xiapu Luo, Hong Kong Polytechnic University, Hong Kong, China, csxluo@comp.polyu.edu.hk; Yan Xiao, Sun Yat-sen University, Shenzhen, China, xiaoyan.hhu@gmail.com; Fei Liu, Shandong University, Qingdao, China, 15066345378@163.com; Yuxuan Zhang, Shandong University, Qingdao, China, 202400130071@mail.sdu.edu.cn; Xiran Lyu, Shandong University, Qingdao, China, 202400130069@mail.sdu.edu.cn; Xianhang Li, Shandong University, Qingdao, China, 202400130007@mail.sdu.edu.cn; Xuanyu Zhu, Shandong University, Qingdao, China, zhuxuanyu@mail.sdu.edu.cn; Chengyi Wang, Shandong University, Qingdao, China, 202300130150@mail.sdu.edu.cn.



This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2026 Copyright held by the owner/author(s).
ACM 2994-970X/2026/7-ARTFSE042
https://doi.org/10.1145/3797137

we analyze 4,831 failed cases from GPT-4o, one of the State-of-the-Art (SOTA) LLMs, revealing numerous third-party reference errors that were obscured previously. These findings highlight the unique challenges of library-centric translation and provide practical guidance for improving TPL-aware code intelligence.

CCS Concepts: • **Software and its engineering → Empirical software validation**.

Additional Key Words and Phrases: Third-party Libraries, Code Translation, Large Language Models, Benchmark

ACM Reference Format:
Pengyu Xue, Kunwu Zheng, Zhen Yang, Yifei Pei, Linhao Wu, Jiahui Dong, Xiapu Luo, Yan Xiao, Fei Liu, Yuxuan Zhang, Xiran Lyu, Xianhang Li, Xuanyu Zhu, and Chengyi Wang. 2026. TransLibEval: Demystify Large Language Models' Capability in Third-Party Library-Targeted Code Translation. *Proc. ACM Softw. Eng.* 3, FSE, Article FSE042 (July 2026), 23 pages. https://doi.org/10.1145/3797137

1 Introduction

Automated code translation has become increasingly important for modern software engineering, as it enables developers to port codebases across multiple Programming Languages (PLs) to adapt to diverse platforms, ranging from cloud services and web applications to embedded systems. In recent years, LLMs, such as GPT-4o and DeepSeek-V3, have been widely studied and applied in code translation tasks across diverse contexts from standalone functions [51, 54, 73, 76] and class-level designs [67] to repository-level environments [24, 37].

However, most existing benchmarks remain limited in their coverage of both the variety and scale of Third-Party Libraries (TPLs), owing to the reasons we analyzed in Section 2, which significantly restricts the exposure of translation errors rooted in external TPLs. Nonetheless, in real-world programming, over 90% of development scenarios necessitate the involvement of TPLs [23], including tasks of data processing, machine learning, web application development, and database interaction. Translating code across languages in these scenarios introduces additional challenges, such as recognizing the correct TPLs, adapting cross-language library mappings, and handling library-specific APIs/parameter conventions. Current benchmarks without limited TPLs cannot adequately reflect these practical challenges. This motivates the creation of a new benchmark that directly measures LLMs' ability to handle external TPLs in code translation tasks.

Benchmark *TransLibEval*: To mitigate the above limitations, we propose *TransLibEval*, the first benchmark specifically designed for evaluating code translation with TPLs. *TransLibEval* consists of 200 real-world translation tasks across Python, Java, and C++, where every task explicitly involves TPLs, covering ten categories such as data processing, utilities, web development, machine learning, and security. Each task is constructed from Python as the calibration PL, then manually translated into Java and C++ by experienced software engineers following strict style and type conventions. *TransLibEval* features (1) more lines of code than most parallel benchmarks with an 88% lift on average. (2) With 5 tests per task, *TransLibEval* achieves 95% statement coverage, significantly improving evaluation efficiency; (3) Existing benchmarks exhibit extremely low library dependency rates (mostly <11%), whereas *TransLibEval* covers 200 libraries across 10 categories, effectively filling a research gap.

Empirical Study: Based on *TransLibEval*, we conduct the first large-scale evaluation of recent LLMs on library-centric code translation. Specifically, we assess seven representative LLMs, including commercial (GPT-4o, GPT-3.5, DeepSeek-V3, Qwen-Max), general-purpose (Llama3-8B/70B), and code-specialized (CodeLlama) models. To capture translation behaviors, we investigate six translation strategies of three families: (1) *Direct translation*, where LLMs translate code snippets with TPLs end-to-end; (2) *IR-guided translation*, which generates intermediate representations (e.g., pseudocode, reasoning traces, or summaries) to guide the final output; and (3) *Retrieval-augmented translation*, where external knowledge from Stack Overflow [52] posts is retrieved to aid library

selection and API usage. We evaluate each program with Compilation Success Rate (CSR), Computational Accuracy (CA), Pass Rate (PR), and Library Dependency Awareness (LDA) to measure whether correct third-party libraries are identified and used.

Main findings: Our experiments reveal several insights. (1) LLMs perform significantly worse on library-centric code translation than on library-free benchmarks, with CA dropping by over 60% on average, highlighting the difficulty of handling TPLs. (2) Commercial LLMs outperform smaller LLMs, with GPT-4o and DeepSeek leading across most metrics, while smaller models like CodeLlama show limited performance despite code specialization. (3) Translations into Python are consistently easier for LLMs, achieving over 50% CA, while translations into Java and C++ remain highly challenging due to stricter type systems and fewer available TPLs. (4) Strategy selection matters: *IR(pseudocode)* performs best when translating into Python, *RA(name)* is more effective for C++, while *RA(method)* and *Direct* excel in Java translations. (5) Error analysis uncovers a great amount of TPL reference errors, where API-related errors, especially calling an irrelevant API, account for over 50% of the total share, while different errors also skewed to diverse translation strategies (see details in Section 5.4). The contributions of this work are threefold:

(1) We construct the first library-centric code translation benchmark, *TransLibEval*, covering 200 real-world TPL-based tasks of 10 categories across Python, Java, and C++. The benchmark dataset and associated code are available at [3].

(2) We provide a systematic evaluation of diverse LLMs and translation strategies, combining correctness, compilation, and dependency-awareness metrics.

(3) We perform a large-scale error categorization of 4,831 failed cases, supported by our detailed 21-page Failed Cases Report [5]. Besides, failures of each translation strategy are also revealed, offering new insights into the challenges of TPL translation and directions for future improvements.

2 Related Work

2.1 Large Language Models for Code Translation

In recent years, substantial effort has been dedicated to investigating and enhancing LLM-based code translation. Initial studies focused on analyzing LLM performance and error types [18, 41], while others explored methods to improve translation fidelity, such as leveraging test cases (UniTrans [73]), employing multi-agent fixing (TRANSAGENT [77]), or developing specialized models (SteloCoder [39]), integrated API knowledge [62], and general correctors (Rectifier [74]) for algorithmic function-level translation.

Beyond standalone functions, recent work has begun to explore code translation at the repository level, addressing real-world complexities like inter-file dependencies and larger codebases [20, 24, 38, 63, 77, 80]. This includes efforts to augment LLMs with extra knowledge, e.g., K^3Trans [38], and with multi-agent systems, e.g., RepoTransAgent [77]. However, owing to the high complexity of code repositories, the studies above are either constrained in repository amount ($\leq$6) [63, 77] or even proactively exclude TPLs to reduce translation difficulties [24, 77]. Certain studies focus on domain-specific PLs (e.g., Rust or Swift), restricting the spectrum of library categories to limited areas [20, 38, 80]. Thus, their empirical evaluations often do not specifically focus on a broad and diverse range of TPL integrations. This leaves a notable gap in understanding LLM capabilities in TPL-targeted code translation scenarios, which account for over 90% in real-world development [23]. As such, this work introduces *TransLibEval*, a benchmark specifically engineered to assess code translation performance on library-centric tasks.

Table 1. Comparative Analysis of Code Translation Benchmarks

Benchmark	Year	Parallel Programming Languages	Practical	Granularity	#Tasks	#Tests/T	#LOC/T	#Tokens/T	#LD	LD%
CodeNet [44]	2021	C,C++,Go,Java,Python	N	Statement/Method-level	200	1.0	34.9	112.0	21	10.5%
AVATAR [7]	2021	Java,Python	N	Statement/Method-level	250	25.1	26.9	149.5	51	20.4%
G-TransEval [27]	2023	C++,C#,Java,Python,JavaScript	N	Method-level	400	5.0	14.3	93.6	-	-
CodeTransOcean-MultilingualTrans [70]	2023	C,C++,C#,Java,Python, Go,PHP,Visual Basic	N	Statement/Method-level	44	1.0	23.8	82.1	3	6.8%
xCodeEval [29]	2024	C,C++,C#,Java,Python,Ruby,Go, JavaScript,Kotlin,PHP,Rust	N	Statement/Method-level	226	46.8	30.8	95.5	27	11.9%
Yang et al. [73]	2024	Python,Java,C++	N	Method-level	568	6.2	12.4	95.8	-	-
PolyHumanEval [54]	2024	Python,C++,C#,Dart,Go,Java,JavaScript,PHP, Kotlin,Ruby,Rust,Scala,Swift,TypeScript	N	Method-level	164	8.1	9.6	37.9	7	4.3%
ClassEval-T [67]	2025	Python,Java,C++	Y	Class-level	94	33.8	66.7	199.5	74	78.7%
***TransLibEval* (Ours)**	**2025**	**Python,Java,C++**	**Y**	**Method-level**	**200**	**5.0**	**41.0**	**120.0**	**200**	**100%**

Note: #Tests/T = Test case number per task; #LOC/T = Line number of code per task; #Tokens/T = Code token number per task; #LD = Task number with library dependencies; LD% = Percentage of tasks with dependencies.

2.2 Existing Benchmarks for Code Translation

Code translation techniques constitute an efficient process for cross-language codebase migration, which is inseparable from a complete evaluation benchmark providing parallel corpora across diverse PLs with accompanying test suites for validation. Early examples include *CodeNet* [44], *AVATAR* [8], *G-TransEval* [26], and *TransCoder-test* [48, 73], sourced from programming contest sites. To further extend the evaluation scope and complexity, *CodeTransOcean* [69] offers a large-scale multilingual dataset of popular and niche PLs with over 7,000 samples, although only 44 algorithmic tasks simultaneously cover all parallel PLs [67]. *xCodeEval* [34] is an execution-based, multilingual benchmark derived from Codeforces [1] supporting both code generation and translation tasks. *PolyHumanEval* [55] extends *HumanEval* [12] to 14 PLs, and *ClassEval-T* [67] introduced class-level translation tasks, incorporating field, method, and library dependencies.

Later on, benchmarks addressing real-world repository-level translation have been proposed. Ibrahimzada et al. [24] and *RepoTransBench* [64] evaluate LLMs on entire repositories but proactively exclude TPLs to alleviate translation difficulties. *RustRepoTrans* [37] is the first function-level code translation benchmark targeting Rust with the context of corresponding repositories. Although comprising 375 tasks, it lacks parallel corpora across all PLs, limiting its evaluation utility for cross-lingual generalization and transfer. *C2R-Bench* [63], Gao et al. [20], and Zhou et al. [80] concentrate on domain-specific PLs migration, including Rust for system programming and Swift/ArkTS for mobile applications, significantly limiting the coverage of TPL categories. Table 1 provides a comparative overview of these benchmarks against *TransLibEval*, detailing their supported PLs, practical alignment, granularity, task amount (**#Task**), test amount (**#Tests/T**), sample scales (**#LOC/T** and **#Tokens/T**), and TPL amount (**#LD**)/percentage (**LD%**). Considering the inherent flaws of non-parallel benchmarks mentioned in Xue et al. [68], which always happen at the repository level, only benchmarks with parallel PLs are included for comparison. As can be seen, (1) most benchmarks carry limited code scale and complexity (e.g., LOC of 14.3 for *G-TransEval*, 9.6 for *PolyHumanEval* on average). Except *ClassEval-T*, a class-level benchmark, *TransLibEval* surpasses others by 88% on average. (2) Although *TransLibEval* contains only 5 test cases per task, lower than most counterparts, its statement coverage can reach 95%, significantly saving the evaluation cost. (3) More importantly, the proportion of tasks involving library dependencies (**LD%**) is strikingly low across most benchmarks (e.g., 10.5% in *CodeNet*, 6.8% in *CodeTransOcean*, and 4.3% in *PolyHumanEval*). *ClassEval-T* contains a relatively high percentage of library dependencies, but the scale is limited to 74, which is less than half of *TransLibEval*. In contrast, *TransLibEval* includes 200 libraries across 10 categories (shown in Table 2), making it an effective code translation benchmark to fill the significant gap of previous related studies.

3 New Benchmark: *TransLibEval*

In the code translation field, a multi-PL parallel benchmark is always constructed from a calibration PL, based on which to extend to other counterpart PLs manually [55, 67, 73]. Considering the extensive Python third-party libraries and their huge community support, we select Python as a calibration PL for benchmark construction, thereby facilitating the library selection and debugging. Additionally, following most code translation settings [24, 26, 67, 73], C++ and Java are chosen as the counterpart PLs in this work. We detail the construction of the calibration and counterpart portion of *TransLibEval*, respectively.

3.1 Calibration Construction

To construct a Python calibration, we first identify Python libraries based on their usage frequency in PyPI download statistics [19] (2020-2024), such as data processing (e.g., numpy, pandas), machine learning (e.g., scikit-learn, lightgbm), web development (e.g., requests, beautifulsoup4), and visualization (e.g., matplotlib, seaborn). For each of the above Python libraries, we formulate four method-level tasks to implement its most typical functionalities. In addition, we select the method-level granularity because it can isolate most external influences (e.g., cross-file/class/method dependencies), making our experiments exclusively focused on LLMs' capability on TPL-targeted code translation. To make the whole implementation more canonical, we follow a series of principles according to previous studies [17, 67]. **(1) For naming conventions.** We follow an officially recommended Python style guide (*PEP 8* [42]) to name identifiers, where methods and variables are named in snake_case (e.g., `compute_area`), classes are required to use the pascal case (e.g., `AreaCalculator`), and constants follow the screaming snake case style (e.g., `MAX_ITERATIONS`). **(2) As for the implementation layout.** Considering that the follow-up translation involves Java programs, whose methods cannot be independently executed without a wrapped class. We require that each Python task must define exactly one top-level class encapsulating a single instance method calling a designated third-party API to exercise library-specific operations, thereby making the parallel corpora more consistent across coding styles. Besides, we use 4 spaces per indentation level according to *PEP 8*, and comments are excluded to avoid additional hints for LLMs when translation. **(3) Towards type conventions.** Following Du et al. [17], we require that all method parameters and return values shall be restricted to primitive data types (`int`, `float`, `boolean`, `string`, etc.). Composite/Object-level data types (e.g., matrices, custom objects) are explicitly prohibited. This constraint guarantees that when invoking focal methods externally from test suites, only limited data types need to be considered across diverse PLs, facilitating the test suite construction and the whole evaluation procedure. **(4) For test suite construction.** We employ Python's *unittest* framework [45] to construct method-level test cases, utilizing its comprehensive assertion APIs and test fixtures such as `setUp()` for pre-test initialization to preserve five essential test categories, including normal operation with valid inputs, edge case evaluation, exception handling scenarios, input type validation, and resource constraint testing. Adhering to the above established principles, two software engineers with ≥ 3 years of professional expertise in Python, Java, and C++ accomplish the above calibration construction, where one is responsible for the specific implementation, while the other's duty is code review and correctness verification. The detailed construction procedures along with a full-workflow example are provided in our repository for reference.[3] Finally, we select 53 libraries to implement their typical functionalities, obtaining 212 code samples, which cost a total of 300 person-hours. The specific statistics are shown in Table 2.

Table 2. The Categorization of Studied Third-Party Libraries for Python, Java, and C++

Category	Description	Examples-Python	Examples-Java	Examples-C++	#Tasks
Data Processing	Numerical computation, and algorithm implementation.	numpy, pandas	JDBI, Jackson	boost, Eigen	68
Utilities	Tools for common utilities, formatting, and extensions.	tqdm, Guava	Commons IO	boost	36
Machine Learning	Libraries supporting ML models' training and inference.	scikit-learn, lightgbm	deeplearning4j, Smile	xgboost, fasttext	28
Web Development	Tools for networking and web application development.	requests, beautifulsoup4	OkHttp, Spring Context	libcurl	24
Visualization	Libraries for plotting charts and visualizing data.	matplotlib, seaborn	JFreeChart	plplot	20
NLP	Libraries for text processing, linguistic analysis, and NLP tasks.	nltk	Stanford CoreNLP	cryptopp	20
Graphics	Libraries for image processing and computer vision.	opencv, PIL	Lucene	opencv	8
Database	Tools for database connection, ORM, and SQL operations.	peewee	JDBI	sqlite3	4
Security	Encryption, hashing, and security-related functionalities.	pycrypto	BouncyCastle	cryptopp	4
Other	Miscellaneous libraries that do not fit into the above categories.	jsonschema	Commons Lang	-	4

3.2 Counterpart Construction

To construct the counterpart portion of *TranLibEval* based on the above calibration, i.e., manually translate the above Python corpus to Java and C++ sides, we strictly follow the five principles established by Xue et al. [67] to carry out the whole translation process. For example, the naming convention (Principle 1) and implementation layout (Principle 3) of translated Java and C++ programs follow their officially recommended style guides [46, 53]. In the type convention aspect (Principle 2), they also defined specific data type mapping rules between Python and Java/C++. From the library selection (Principle 4) and test suite construction (Principle 5) perspectives, explicit library searching workarounds, library management tools (*maven* [10] for Java and *nuget* [36] for C++), and testing frameworks (*JUnit* [28] for Java and *GoogleTest* [22] for C++) are also designated.

Specifically, five software engineers with ≥ 3 years of professional expertise in Python, Java, and C++ are involved. The lead architect assumed ultimate responsibility for review and arbitration, while the remaining members formed two PL-specific expert groups. Each group is responsible for either Java or C++-oriented translation and is composed of two members, one is a primary translator and the other is a dedicated verifier. The primary translator adheres to the five principles established by Xue et al. to translate Python to Java/C++, including both the TPL-related programs and their corresponding test suites. Afterwards, for each translation sample, the verifier first exhaustively reviews the translated result, examining whether it conforms to the five principles, and then executes these test cases to double-check the correctness of the translation. Among the five principles, disagreements between translators and verifiers normally arise in type handling (Principle 2), library selection (Principle 4), and test equivalence (Principle 5). At this point, the final decision is made by lead architect. After the aforementioned manual translation process, 12 samples still cannot reach a consensus, even with the engagement of the lead architect, owing to their translation difficulties. For example, *spaCy* and *TensorFlow* are two of this kind, where the former lacks any Java/C++ equivalent while the latter exhibits significant API divergence across PL bindings. We exclude these 12 samples and finally yield a parallel TPL-targeted code translation benchmark with 200 samples. The entire construction consumed 200 person-hours.

4 Experimental Design

This study utilizes the *TransLibEval* benchmark to evaluate recent LLMs on library-centric code translation. We aim to address the following research questions (RQs):

RQ1 (Overall Correctness): How do recent LLMs perform on library-centric code translation? Compared with general code translation, library-centric translation introduces additional challenges due to the need for correct recognition, import, and use of third-party libraries. Given the increasing prevalence of external library APIs in modern software development, it becomes critical to understand how well LLMs can handle such dependencies across languages.

RQ2 (Translation Strategies): How do different translation strategies affect the performance of recent LLMs on library-centric code translation? Developers may adopt different

prompting strategies like direct translation, Intermediate Representation (IR)-guided translation, or retrieval-augmented translation. Evaluating how these strategies impact the handling of library dependencies can offer practical guidance for prompt engineering and model deployment.

RQ3 (Library Dependency Awareness): How do the LLMs perform in identifying necessary and available libraries? Library-centric code translation often requires not just syntax conversion but also awareness of the necessary libraries for implementing equivalent functionalities. Considering that the library mappings between different PLs are not always one-to-one, LLMs have to infer a limited library set to accomplish the translation task in this work. Nonetheless, LLMs' above-mentioned ability is still unknown, and the corresponding investigation has not been extensively conducted before, thus motivating us to delve into this RQ.

RQ4 (Failed Cases Analysis): What kind of errors do LLMs make in library-centric translation, and how frequent are they? While prior work has examined LLM failures in method-, class- or even repo-level translation [37, 40, 54, 66, 73], their included libraries, even in repo-level, are limited as shown in Section 2. Thus, a large-scale fine-grained analysis of library-related errors is still lacking, and identifying these failure modes can help the community develop more targeted improvements for LLM-based code translation systems.

4.1 Studied LLMs

Table 3 lists our studied LLMs with their categories, base model, released times (Time), parameter sizes (Size), whether they utilize Mixture-of-Experts (MoE) and Rotary Position Embedding (RoPE) techniques, pretraining token numbers (Training Base), and the input/output context window token limits (In/Out). For those models whose context windows are measured in characters, we specified them separately. As can be seen, to extensively explore the performance of the latest LLMs, we select LLMs from general (i.e., Llama 3 [33] and Gemma [57]), code (i.e., CodeLlama [51]), and commercial kinds [1] (i.e., DeepSeek-V3 [15], GPT-4o [4], GPT-3.5-Turbo and Qwen-Max [58]), released from late 2023 to late 2024. Furthermore, for LLMs from the same family, we also select two different sizes (i.e., Llama3-8B/70B) to study the influence of the volume of model parameters.

Table 3. Studied LLMs in this paper

Category	Model	Base Model	Time	Size	MoE	RoPE	Training Base (Tokens)	In/Out (Tokens)
Commercial	DeepSeek-V3 [15]	-	2024-12	671B	Yes	Yes	14.8 trillion	64k / 4096
	GPT-4o [4]	-	2024-05	~200B [6]	Yes	Yes	-	128k / 4096
	GPT-3.5-Turbo	-	2023-03	-	No	Yes	-	4k / 4096
	Qwen-Max [58]	-	2025-01	-	Yes	Yes	20 trillion	30k / 8192
General	Llama3 [33]	-	2024-04	8B	No	Yes	15.6 trillion	7k / 1024
	Llama3 [33]	-	2024-04	70B	No	Yes	15.6 trillion	7k / 1024
Code	CodeLlama [51]	Llama2 [61]	2023-08	7B	No	Yes	500 billion	8000 Characters/ 1024

In the following sections, "DeepSeek" refers to DeepSeek-V3, "Qwen" indicates Qwen-Max, and "GPT-3.5" indicates GPT-3.5-Turbo. We also use "Llama3-70B" and "Llama3-8B" to denote the two versions of Llama3.

4.2 Studied Translation Strategies

Given a library-centric code translation task, we investigate the following six translation strategies of three categories for each of the studied LLMs.

(1) Direct Translation (Direct): LLMs are provided with the raw source-language code snippet involving third-party library usage and are asked to translate it into the target PL directly. This strategy simulates the most straightforward application of LLMs in real-world scenarios where

[1]Following [67], commercial LLMs are defined as large language models with substantial parameter sizes (typically exceeding 100B), developed by organizations for commercial purposes, and providing publicly accessible interfaces for interactive dialogue and text-based applications.

developers expect end-to-end translation without any auxiliary reformulation or decomposition of the input. A prompt example is shown in [3].

(2) IR-Guided Translation (IR): IRs, such as pseudocode, reasoning traces, or summaries, have been widely used in LLM-based code intelligence [25, 30, 78]. Its core idea is requiring LLMs to generate an IR first, as a more explicit and specific instruction, to guide them in producing the final code snippets. Apart from the above, we consider that IR also has the potential to isolate the influence of source PLs, making LLMs focus more on the functionality implementation. This is a critical motivation in library-centric code translation, as many previous studies [40, 71] have revealed that LLMs tend to copy APIs of source PLs, which could be more severe in our library-intensive setting. Below, we introduce three typical IR-based prompting methods for this study. Prompt examples of each variant are shown in [3].

- *IR(CoT)*: It first requires LLMs to break down the program execution logic of source PL into a sequence of natural language reasoning steps. Then, LLMs are guided to translate the program into the target PL version based on the above IR prompt.
- *IR(pseudocode)*: It replaces the above natural language reasoning sequences with pseudocode as the IR between source and target PLs, where the pseudocode consists of essential program elements, such as branches, loops, and hierarchical structures.
- *IR(summary)*: It first requires LLMs to generate a summary for the program to be translated, where the summary contains the core functionality and input/output requirements. Subsequently, LLMs are prompted to translate the program of the source PL to its counterpart in the target PL based on the above summary.

(3) Retrieval-Augmented Translation (RA): Owing to the complex library mapping relation among diverse PLs, previous studies [40, 67] have noticed that correctly identifying and applying necessary libraries in the target PL when translating code is critically difficult for current LLMs. To address this challenge, an intuitive direction is to inject external knowledge into LLMs via retrieval, thus RA becomes another kind of highly motivated method to be studied in library-centric code translation. Specifically, we retrieve relevant external knowledge from Stack Overflow [52] to augment LLMs' capability. We explore two variants, and their prompt examples are shown in [3].

- *RA(method)*: It instructs LLMs to analyze the entire function and generate a dedicated search query for code translation. Subsequently, based on Stack Exchange API [2], we develop a retrieval tool that searches on Stack Overflow with the above-generated query, and returns the top-k most relevant posts. For each post, the retrieval tool fetches the top-n most upvoted answers for prompting LLMs to identify specific library/API usage and generate the final translation results. We set k=1 and n=3 in this study.
- *RA(name)*: Method names normally convey program functionalities more succinctly compared with RA(method)'s query. As an alternative to *RA(method)*, *RA(Name)* uses only the target PLs plus method names (e.g., `Python: "image_to_grayscale"`) as queries to retrieve external knowledge. Other settings and procedures are all the same as *RA(method)*.

4.3 Evaluation Metrics

We follow previous studies [49, 50, 73] and evaluate LLMs via Compilation Success Rate (CSR), Pass Rate (PR), Computational Accuracy (CA), and Library Dependency Awareness rate (LDA).

Compilation Success Rate (CSR): It computes the ratio of samples that can be successfully compiled after translation. We define CSR as below.

$$CSR = \frac{\sum_{k=1}^{N} cs(\hat{y_k})}{N}, \ \textbf{where } cs(\hat{y_k}) = \begin{cases} 1 & Compile(\hat{y_k}) \rightarrow success \\ 0 & Compile(\hat{y_k}) \rightarrow error \end{cases} \tag{1}$$

where N denotes the total number of samples, $\hat{y_k}$ denotes the k-th translated sample via a certain LLM. *Compile(·)* denotes compiling samples with their corresponding compilers, such as *javac* for Java while *g++* for C++, and its results are either *success* or *error*.

Pass Rate (PR): It computes the ratio of successfully passed test cases across all samples, where a passed test case means the execution result of a translated program equals that of the ground truth program. We define PR as below.

$$PR = \frac{\sum_{k=1}^{N}\sum_{j=1}^{T_k} ps(\hat{y_k}, y_k, j)}{\sum_{k=1}^{N} T_k}, \textbf{ where } ps(\hat{y_k}, y_k, j) = \begin{cases} 1 & Exec_{k,j}(y_k, j) = Exec_{k,j}(\hat{y_k}, j) \\ 0 & Exec_{k,j}(y_k, j) \neq Exec_{k,j}(\hat{y_k}, j) \end{cases} \tag{2}$$

where T_k is the total number of test cases for the k-th sample, and y_k denotes the ground truth of the k-th sample. $Exec_{k,j}(\cdot)$ denotes the execution result of a program (y_k or $\hat{y_k}$) on the j-th test case from the test suite of the k-th sample.

Computational Accuracy (CA): It computes the ratio that the translated programs can produce the same execution result as the ground truths, given a set of test suites. We define CA as below.

$$CA = \frac{\sum_{k=1}^{N} ca(y_k, \hat{y_k})}{N}, \textbf{ where } ca(y_k, \hat{y_k}) = \begin{cases} 1 & Exec_k(y_k) = Exec_k(\hat{y_k}) \\ 0 & Exec_k(y_k) \neq Exec_k(\hat{y_k}) \end{cases} \tag{3}$$

where $Exec_k(\cdot)$ denotes the execution result of a program with the test suite of the k-th sample.

Library Dependency Awareness Rate (LDA): To assess whether LLMs can correctly identify useful third-party libraries when translation, we adapt the dependency recall metric from prior works [17, 75]. Specifically, LDA measures the ratio of translated programs that invoke useful third-party libraries to fully achieve the intended functionalities, which can be formulated as below.

$$LDA = \frac{N_{lib}}{N} \tag{4}$$

where N_{lib} denotes the number of translated samples that invoke useful libraries. We delineate the specific definition of invoking useful libraries and the counting methodology of N_{lib} in Section 5.3.

4.4 Implementation Details

We utilize seven LLMs for our experiments. Specifically, we employ the Deepseek, Qwen-Max, GPT-4o, and GPT-3.5 via their official API interface[4, 14]. Additionally, CodeLlama, Llama3-8B, and Llama3-70B are operated via the open API interface provided by Baidu Qianfan platform[11]. The hyperparameters for the code translation process are set as follows: $temperature = 0$ and $n = 1$, which means we only fetch LLMs' first translated candidates for evaluation during inference, and the LLMs' randomness can be eliminated to a minimum, thereby ensuring the reproducibility of the experiments. Nonetheless, despite this, the randomness persists to some extent. Thus, we assessed stability by repeating a subset of experiments three times following previous studies [60]. This subset specifically comprised C++ to Python translations using the Direct strategy. The results showed that all LLMs except GPT-3.5-Turbo exhibited a Pass Rate variance of less than 5%, indicating consistently stable behavior. GPT-3.5-Turbo showed a deviation of 5.9%. The overall low variance observed in this analysis suggests that the inherent randomness had a negligible influence on the reliability of our conclusions for the study as a whole. All other hyperparameters are kept by default. All evaluations are performed in a one-shot setting.

5 Experimental Results

5.1 RQ1: Overall Correctness

To intuitively understand the difficulty and motivation of library-centric code translation tasks, we evaluate each studied LLM on both the *TransLibEval* and the benchmark released by Yang et al.[73], where the latter is also a method-level code translation benchmark but TPL-free, thereby making a direct comparison. Figure 1 demonstrates their correctness comparison in terms of average Compilation Success Rate (CSR), Computational Accuracy (CA), and Pass Rate (PR). Besides, Table 4 presents the detailed experimental results on *TransLibEval*. It should be noted that the Direct translation strategy is used for LLMs in RQ1, and we have the following observations.

Library-free vs. Third-party library-centric Translation. As shown in Figure 1, the code translation performance of LLMs degenerates significantly and consistently on all translation pairs when converting from library-free to TPL-centric benchmark. Specifically, all LLMs studied can achieve CSR over 95%, CA over 78% and PR over 76% on almost all translation pairs in Yang et al.'s benchmark. However, when assessing them on the *TransLibEval*, LLMs' code translation performance declines by 41.63% on average in terms of CSR, 61.03% on average in terms of CA, while by 54.49% on average in terms of PR. The most challenging translation pairs, Python-to-Java and Python-to-C++, suffer catastrophic declines of 61.92% and 63.16% in CSR, 82.96% and 73.48% in CA, and 75.83% and 76.24% in PR, respectively. The dramatic performance degradation can be attributed to the inherent complexity of involving practical TPLs in code translation, which involves handling intricate API dependencies, complex type systems, and sophisticated library-specific semantics that are absent in library-free code translations.

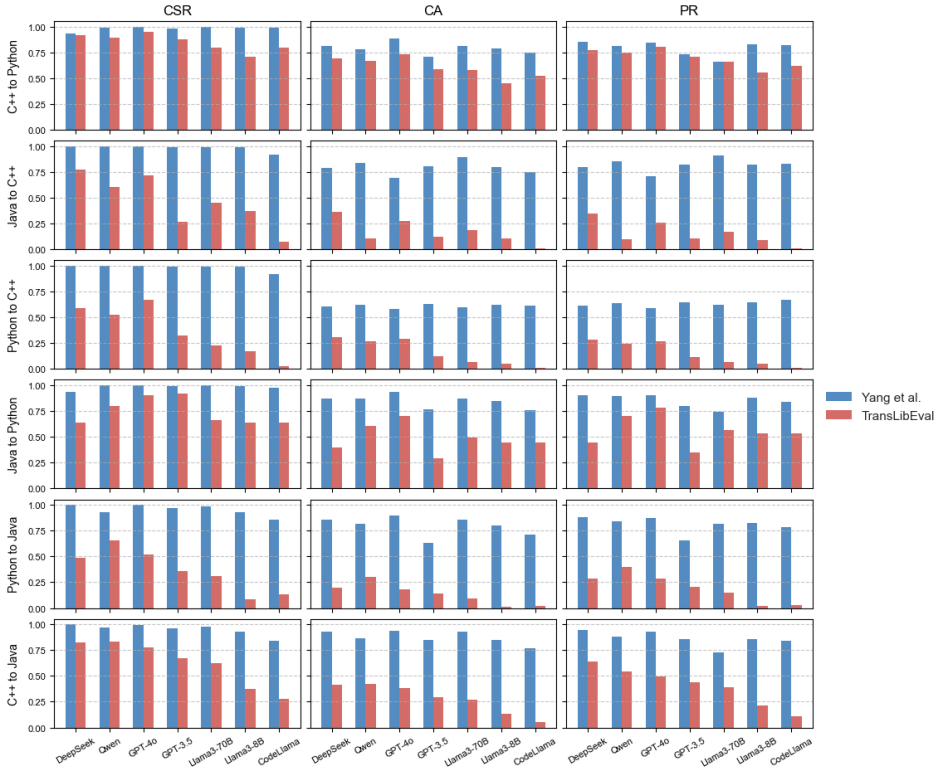


Fig. 1. Performance Comparison between *TransLibEval* and Yang et al. among Diverse LLMs

Finding 1: Compared with method-level code translations, LLMs show a marked decline in performance when translating code involving TPLs, with Python-to-Java and Python-to-C++ translations suffering the most severe degradation (e.g., over 80% in CA), primarily due to the complex API dependencies and library-specific semantics.

Comparison among LLMs. Focusing on the results of the *TransLibEval* benchmark (see Table 4), commercial LLMs (GPT-4o, GPT-3.5, DeepSeek, and Qwen) consistently perform the best, achieving average scores of 68.56% in terms of CSR, 36.94% in terms of CA, and 43.05% in terms of PR across various translation pairs. Commercial LLMs demonstrate a significant advantage over smaller LLMs, with relative improvements of 67.79% in CSR, 68.11% in CA, and 62.59% in PR. Among commercial LLMs, GPT-4o achieves the highest performance with 75.42% in terms of CSR, 42.67% in terms of CA, and 48.28% in terms of PR, followed closely by DeepSeek and Qwen. This superiority can be attributed to their more sophisticated architecture, larger parameter sizes, and extensive training on diverse code corpora that include TPL usage patterns. Among smaller LLMs, Llama3-70B demonstrates the strongest overall performance, yet it still lags considerably behind leading commercial counterparts. Notably, compared with Llama3-8B, CodeLlama performs weakly overall, although their parameter sizes are almost the same, and the latter was pre-trained on code. We infer the reason is: CodeLlama is built upon the Llama2 [61], whose training base is 2 trillion tokens, which only accounts for one seventh of Llama3-8B's training base, as shown in Table 3.

Finding 2: Commercial LLMs exhibit predominantly superior performance on code translation involving TPLs, significantly outperforming smaller LLMs by 67.79% in CSR, 68.11% in CA and 62.59% in PR. LLMs with limited training bases or smaller sizes always perform more weakly.

Comparison among translation directions. Based on Figure 1 and Table 4, LLMs generally perform much better on Python-oriented translations (e.g., C++/Java-to-Python) in terms of all evaluation metrics. Python-oriented translations achieve average scores of 54.39% in terms of CA, 79.46% in terms of CSR, and 62.64% in terms of PR, representing substantial advantages of 192.60%, 75.40%, and 177.11% in terms of each metric in order compared to other translation directions. However, concerning translations to C++ and Java, LLMs' performance drops dramatically, particularly, Python-to-C++ and Python-to-Java show the most significant performance degradation, with CSR falling below 37%, CA falling below 17% and PR below 15%. The performance differences can be attributed to both pre-training data biases and the inherent complexity of different PLs. Firstly, the superior performance in Python-oriented translations aligns with Python's dominance in open-source version control platforms (e.g., GitHub) since 2022 [21], which are the most prevalent code databases for LLMs' pre-training [31, 51, 54, 56, 79]. Secondly, Python, as an interpreted PL with a dynamic type system, carries more human-readable syntax in library usage, making it analogous to natural language and facilitating LLMs' learning [65]. In contrast, LLMs face greater challenges when dealing with the strict type systems, complex memory management requirements, and intricate library interfaces of C++ and Java in third-party library contexts [43].

Finding 3: LLMs manifest substantial superiority in translating other PLs to Python within TPL contexts, while struggling significantly with translations to C++ and Java (e.g., 54.39% v.s. 18.59% in CA) due to both the training base and language characteristics.

5.2 RQ2: Translation Strategies

Our studied code translation strategy can be divided into three categories, i.e., direct, IR-based, and retrieval-based methods. As introduced in Section 4.2, we propose three IR-based methods in

Table 4. Experimental Results with direct translation on *TransLibEval*

Models	C++ to Python			Python to C++			Java to Python			Python to Java			C++ to Java			Java to C++			Average		
	CSR	CA	PR	CSR	CA	PR	CSR	CA	PR	CSR	CA	PR	CSR	CA	PR	CSR	CA	PR	CSR	CA	PR
DeepSeek	91.50	69.50	77.60	59.00	31.00	28.60	64.00	40.00	44.50	48.50	20.00	28.70	82.00	41.00	64.00	77.00	36.50	34.70	70.33	39.67	46.35
Qwen	89.00	67.00	75.20	52.50	27.00	24.70	80.00	60.50	69.90	65.50	30.00	39.50	83.00	42.00	54.30	60.50	10.50	9.60	71.75	39.50	45.53
GPT-4o	94.50	73.00	80.80	66.50	29.00	27.10	90.50	70.00	78.20	52.00	18.00	28.01	77.50	38.50	49.50	71.50	27.50	26.00	**75.42**	**42.67**	**48.27**
GPT-3.5	87.50	58.50	70.40	32.50	12.50	11.90	91.50	29.00	34.50	35.50	14.00	20.70	67.00	29.50	43.70	26.50	12.00	11.00	56.75	25.92	32.03
Llama3-70B	80.00	58.00	66.10	23.00	7.00	6.50	66.00	49.50	56.20	30.50	9.50	14.70	62.00	27.00	39.30	45.50	18.50	16.90	51.17	28.25	33.28
Llama3-8B	70.50	45.00	55.30	17.00	5.50	5.20	64.00	44.50	53.10	8.50	1.00	2.20	37.50	13.00	21.50	37.00	10.50	9.00	39.08	19.92	24.38
CodeLlama	79.50	52.50	62.00	2.50	1.00	0.80	64.00	44.50	53.10	13.00	2.00	3.10	27.50	5.50	10.90	7.50	1.00	0.70	32.33	19.91	21.76
Average	**84.64**	**60.50**	**69.63**	36.14	16.14	14.97	74.29	48.29	55.64	36.21	13.50	19.57	62.36	28.07	40.46	46.50	16.64	15.41	55.09	32.19	36.82

$^{\Phi}$ Table cells with light cyan background denote the highest performance in terms of CSR/CA/PR on each translation pair among various LLMs. Bold and underlined values represent the highest and second-highest average performance, respectively.

this work, including *IR(CoT)*, *IR(pseudocode)*, and *IR(summary)*. As for the retrieval-based method, a *RA(method)* and *RA(name)* are proposed for investigation, respectively. Figure 3 presents the performance of various translation strategies in diverse LLMs on *TransLibEval*. The key observations from our analysis are as follows:

C++ as the Target PL: For C++-oriented translation, *RA(name)* and *IR(pseudocode)* strategies are the most effective across almost all evaluation metrics. *RA(name)* excels due to C++'s strong convention of employing explicit, descriptive identifiers that precisely reveal function intent—a practice deeply ingrained in the C++ community for enhancing code readability and maintainability [9, 16]. As shown in Figure 2, through the *RA(name)* strategy, our tool precisely retrieves effective posts from the knowledge base to produce the appropriate C++ implementation, exemplified by the *CalculateDistance()* function. In our experiments, *RA(name)* achieved a 41% PR score, with Qwen performing particularly well in capturing these naming conventions. *IR(pseudocode)* effectively bridges the translation gap from library-rich PLs to library-limited ones. Python and Java often rely on various off-the-shelf APIs to program, while C++ codes need to implement corresponding functionalities owing to the relatively limited libraries. Thus, LLMs tend to directly copy Python/Java APIs and libraries to implement the functionalities of the C++ side. In this case, *IR(pseudocode)* overcomes the above problem by isolating the influence of the original PLs but making LLMs focus exclusively on functionality logic.

Finding 4: For C++-oriented translations, *RA(name)* and *IR(pseudocode)* strategies are the most effective. The intuitive method naming convention in the community facilitates retrieval with *RA(name)* strategy, while *IR(pseudocode)* isolates the API usage of original PLs, alleviating failures induced by library/API copy.

```cpp
// C++ - how to calculate distance between points using vectors

struct Point { float x,y; };
float CalculateDistance(const Point& p1, const Point& p2) {
    return sqrt((p1.x-p2.x)*(p1.x-p2.x)+(p1.y-p2.y)*(p1.y-p2.y));
}
std::vector<double> distances;
for(int a=0; a<frame1.size(); a++) {
    for(int b=0; b<frame2.size(); b++) {
        distances.push_back( dist(frame1.at(a), frame2.at(b)) );
    }
}
```

(a)

```cpp
// android - Can't create handler inside thread that has not called
Looper.prepare()

protected void handleUIRequest(String message)
{
    Message msg = uiHandler.obtainMessage(UIHandler.DISPLAY_UI_TOAST);
    msg.obj = message;
    uiHandler.sendMessage(msg);
}
```

(b)

Fig. 2. Effectiveness of the *RA(Name)* Strategy Across Different Target Languages. (a) Successful retrieval of relevant C++ code patterns for *CalculateDistance()*. (b) Unsuccessful retrieval in Java, returning irrelevant Android-specific code.

Java as the Target PL: Considering that the performance on CSR seems kind of complicated, as we cannot summarize which strategy performs best on Java-oriented translation, and CSR only evaluates on the compilation level, we focus on the evaluation metrics of CA and PR instead here. From this perspective, *RA(method)* and *Direct* strategies prove to be the most effective. *RA(method)*

can leverage rich contextual signals from method calls—such as parameter types, surrounding API sequences, and exception patterns—rather than relying solely on method names to retrieve reference implementations for LLMs. This is crucial in Java's vast and varied ecosystem. For instance, when doing Python-to-Java translation, *RA(name)* searches Java posts on Stack Overflow with a query of *handle_request*, a method name of a web-end Python program. However, an Android-end implementation for handling UI requests as shown in Figure 2(b) was obtained. This contradicts the Python program's intent, which is not for the mobile application, misleading LLMs' translation. As such, Retrieval based on full method context helps avoid such ambiguities and aligns well with Java's heavy reliance on idiomatic API usage. On the other hand, the *Direct* strategy remains effective, especially between Java and PLs with syntactic similarity such as C++. Shared structural conventions and relatively comparable standard libraries enable surprisingly accurate translations without additional retrieval.

Finding 5: For Java-oriented translations, *RA(method)* and *Direct* strategies achieve the best results. The former allows for more contextual semantics to eliminate platform ambiguity, while the latter dominates in the C++ to Java translation owing to syntactic similarity.

Python as the Target PL: For Python-oriented translations, IR-guided strategies perform the best overall. A potential explanation is that the massive training base and language characteristics mentioned in Section 5.1 provide a relatively reliable generative ability for Python programs. Even without proactively injecting external knowledge, LLMs can induce numerous useful libraries for functionality implementation as shown in Table 5, Section 5.3. Among IR strategies, *IR(pseudocode)* ranks the top-2 highest across almost all LLMs and evaluation metrics, because pseudocode as IR carries structured implementation logic for LLMs to follow, allowing LLMs to focus on core logic while abstracting away unnecessary syntactic complexity.

Finding 6: For Python-oriented translations, the IR strategy, especially *IR(pseudocode)*, outperforms all others overall, owing to the advantages of LLMs themselves in Python programming and structured implementation logic offered by *IR(pseudocode)*, making it the top choice for translating into Python.

5.3 RQ3: Library-Dependency Awareness

Apart from the above correctness analysis, this section delves deeper into each translation from a library awareness perspective. Considering that the judgment of whether the LLMs identify useful libraries to accomplish the functionalities in the target PL side is hard to assess automatically, we opt to do so in a manual manner, which is the same as most previous related studies [17, 67]. Following their common workaround, we employed a stratified random sampling approach across all models, translation directions, and strategies. Samples were drawn proportionally from each stratum to form a balanced and unbiased sub-dataset for manual analysis. The data volume is 756, determined with a 95% confidence level and a 5% confidence interval, ensuring robust statistical coverage. To conduct a rigorous manual analysis, we determine that a translation satisfies LDA by examining 1) whether the translated program imports useful libraries, and 2) whether the APIs it invokes from imported libraries can implement the equivalent functionalities. The entire analysis process required a total of 70 person-hours. Four evaluators with 3–5 years of development experience carried out the manual analysis, with each sample being independently rated by two labelers. We employed Cohen's kappa coefficient to evaluate inter-rater agreement, which is especially suitable for our task since the evaluation involves two distinct labels. The resulting kappa score of 0.71 indicates substantial agreement between the raters. To simplify the analysis, Figure 4 displays the LDA results across different LLMs and translation directions. Considering the great correlation

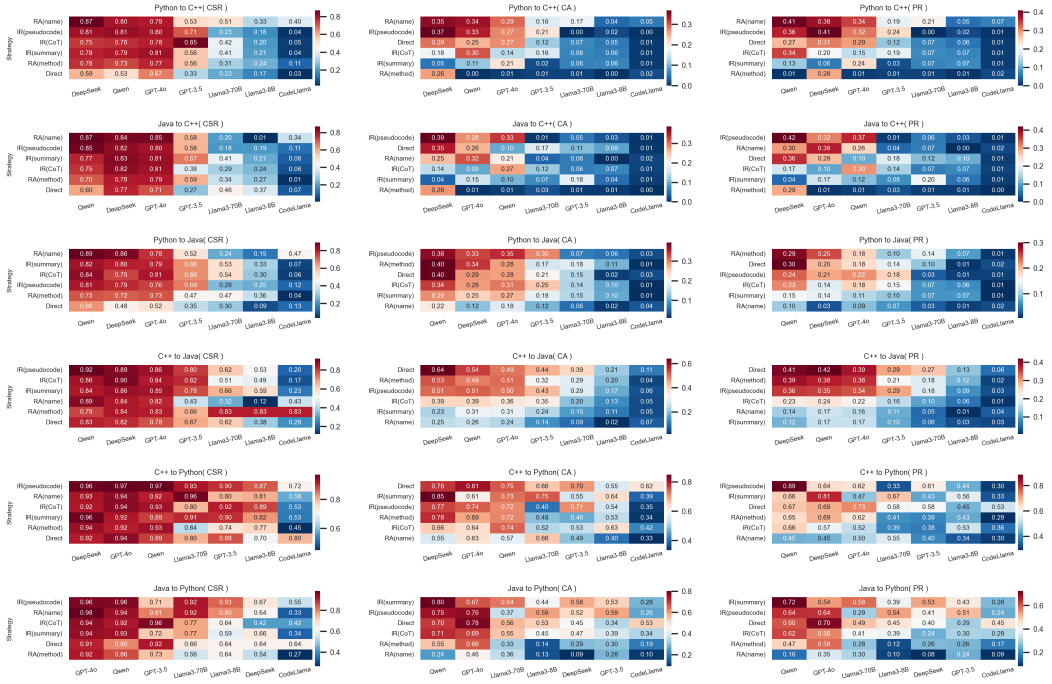


Fig. 3. Performance of Different Translation Strategies on *TransLibEval*

between translation strategies and translation directions during our manual analysis, we present Table 5 to simultaneously examine their TPL awareness performance.

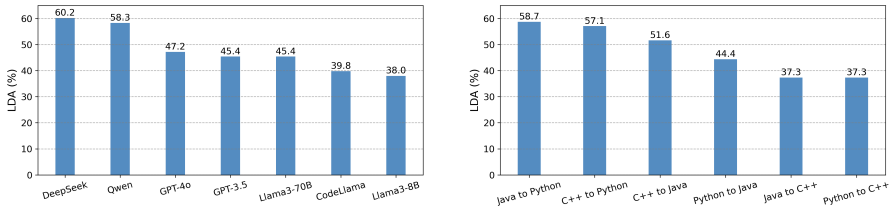


Fig. 4. The Library-Dependency Awareness Analysis across Different LLMs and Translation Directions

From the model perspective, commercial LLMs (e.g., DeepSeek, Qwen, GPT-4o, and GPT-3.5) consistently outperform other alternatives (e.g., Llama3-70/8B and CodeLlama) in identifying useful TPLs and invoking associated APIs by 28.51% on average. In particular, DeepSeek leads with the highest LDA of 60.2%. This trend can be attributed to their broader and more tremendous training bases, enormous parameter sizes, and advanced model architectures (e.g., MOE).

Regarding translation directions, Python-oriented translations (i.e., Java/C++-to-Python) consistently demonstrate the highest LDA of 57.9% on average, due to Python programs' massive training samples, broad availability of standard libraries, and natural language analogous features, lowering the threshold of LLMs to identify useful Python libraries and associated APIs. On the other hand, we also noticed that C++-oriented translations (i.e., Python/Java-to-C++) present more significant challenges for library dependency management, as evidenced by lower LDA scores of 37.3% on average. A reasonable explanation is that standard libraries of C++ are limited to core language support, such as containers and algorithms. Implementing functionalities, such as AES symmetric encryption/decryption [3], requires collaboration of multiple TPLs or even partial implementation from scratch owing to the unavailability of certain libraries, which also significantly aggravates the difficulty of LLMs in identifying useful libraries.

Table 5. Strategy-wise LDA Rate (%) across Translation Directions

Strategy	C++ to Python	Python to C++	Java to Python	Python to Java	C++ to Java	Java to C++
Direct	57.14	38.10	71.43	66.67	66.67	38.10
IR(CoT)	66.67	33.33	47.62	42.86	57.14	23.81
IR(pseudocode)	71.43	33.33	57.14	38.10	47.62	52.38
IR(summary)	57.14	28.57	76.19	42.86	47.62	42.86
RA(method)	33.33	42.86	38.10	38.10	52.38	14.29
RA(name)	57.14	47.62	61.90	38.10	38.10	52.38

Cells with light cyan background denote the highest LDA rate in each direction among all strategies.

In terms of strategies, LDA varies depending on different translation directions, especially the target PLs, as shown in Table 5. For Python as the target, IR-guided strategies, either *IR(pseudocode)* or *IR(summary)*, achieve the highest LDA scores, outperforming other counterparts by 31.60%–37.93%. When translating to C++, *RA(name)* consistently keeps the best performance on library usage, surpassing counterparts by 35.14% on the Python-to-C++ translation. As for Java-to-C++ translation, *RA(name)* ties with *IR(pseudocode)* but still excels other strategies by 75.98%. Regarding the Java-oriented translation, *Direct* dominates among various strategies, obtaining a significant lead by 37.26%–66.68%. The above conclusion of the LDA analysis among the different strategies is generally consistent with the correctness analysis across strategies in Section 5.2, demonstrating in a more granular way the importance of identifying useful libraries/APIs in real-world code translation.

Finding 7: Commercial LLMs outperform others in handling TPL dependencies. Python's language features facilitate better dependency handling, while C++ presents greater challenges due to its complexity. Additionally, the choice of strategy significantly affects translation quality and dependency management.

5.4 RQ4: Failed Cases Analysis

This section meticulously examines failure cases that one of the SOTA LLMs (i.e., GPT-4o) made, aiming to identify persistent challenges in code translation involving various libraries, thereby highlighting areas where even the most advanced models require further refinement.

Our methodology here is a rigorous adaptation of the thematic analysis framework successfully employed by *ClassEval-T* [67]. For our evaluation, we dedicated 360 person-hours to thoroughly analyze 4,831 error cases selected from a single-time experiment with GPT-4o. We adopted the same structured, multi-stage process to ensure the accuracy and consistency of our findings. The process began with 3 experts with over five years of experience in Python, Java, and C++ who analyzed an initial sample set to create a comprehensive codebook for error categorization. This guide was then used by 7 experienced evaluators who assessed each case through a double-blind, dual-review process. In instances where evaluations diverged, the evaluators engaged in a structured discussion to reach a consensus, with unresolved disputes being escalated to the expert team for arbitration. To ensure the exhaustiveness of our findings, we conducted the categorization at a statement (e.g., API misuse)/token-level (e.g., indentation or symbol error) granularity, thereby a translation sample may consist of multiple types of errors.

Table 6 presents the evaluation results for 6 translation strategies on *TransLibEval* across 6 translation directions. To better explain and clarify each type of error identified in our taxonomy, we have compiled a detailed 21-page Failed Cases Report, which is available in our repository [5]. From the overall distribution, **third-party reference errors (A)** account for the vast majority of issues, with an average proportion as high as 79.58%, making them the primary cause of translation failure. In contrast, **syntactic errors (B)** and **code generation errors (C)** represent a smaller share, at 6.3% and 11.1% respectively. The proportions for **runtime errors (D)** and **other miscellaneous errors (E)** are very small, at just 2.48% and 0.54%. This clearly strengthens the position of *TranLibEval* in

evaluating library-centric code translation and exposing diverse library-related errors when LLM translates code.

Third-party reference errors primarily involve three aspects: **library-related errors (A-1)**, with an average proportion of 34.41%; **API-related errors (A-2)**, with an average of 50.1%; and **parameter-related errors (A-3)**, at 15.49%. The high proportion of API-related errors indicates that API adaptation is the primary challenge in cross-PL translation, followed by library reference issues. **Library-related errors (A-1)** stem from the improper mapping or handling of source-PL libraries in the target PL. The most common issue within this sub-category is **retaining the original PL's library (A-1-3)**, which accounts for 44.44% of these errors. An example would be keeping `import numpy` in Java code. Furthermore, **calling an invalid library (A-1-1)** that doesn't exist or isn't supported in the target PL accounts for 36.44%, while **missing required libraries (A-1-4)** account for 11.0%, and **missing import statements (A-1-2)** for 8.12%. **API-related errors (A-2)** are more challenging; the most prominent issue is **calling an irrelevant API (A-2-3)**, which makes up 55.31% and often leads to a complete failure in code logic. Additionally, **retaining the original PL's API (A-2-1)** accounts for 33.24%, while **missing an API (A-2-4)** and **calling a non-existent API (A-2-2)** account for 5.79% and 5.66%, respectively. **Parameter-related errors (A-3)** are mainly characterized by mismatches in parameter type, count, or return value when invoking APIs. Within this sub-category, **parameter type mismatch (A-3-1)** is the leading issue, accounting for 39.37%. **Return value type errors (A-3-3)** represent 34.75%, and **parameter count errors (A-3-2)** account for 25.89%.

The existence of syntactic errors indicates that LLMs still have shortcomings in syntax tree mapping between source and target PLs. These errors primarily include three sub-categories: **Syntax structure errors (B-1)**, which account for 38.4% and manifest as basic syntax issues like missing parentheses, improper indentation, or incomplete statements; **Language feature errors (B-2)**, which account for 21.86% and stem from the failure to incorrectly translate unique syntax structures of the source PL (e.g., list comprehensions, pattern matching) into their equivalent expressions in the target PL; and **Object definition errors (B-3)**, which account for 39.73%, referring to syntax errors occur in variable or function definition.

Code generation errors, which account for 11.1% of the total, are centered on the completeness and logical consistency of the output. Within this category, **the generation of non-code content (C-1)** accounts for 4.55%, and the output mainly consists of explanatory text indicating the meaning of the source PL or text indicating that the task cannot be completed. In addition, **incomplete code generation (C-2)** and **contradictory code generation (C-3)** are the main issues, each accounting for approximately half of the cases. This suggests that the LLMs struggle to maintain code integrity at the generation level, especially with longer code or complex logic. **Runtime errors (D)**, which account for 2.48%, are problems that are often difficult to expose during static analysis but appear during actual execution. **Other errors (E)** account for only 0.54%, mainly referring to miscellaneous issues that are difficult to classify and do not fall under major errors.

Finding 8: Third-party reference errors (A) are the dominant cause of code translation failures, accounting for a massive 79.58% of all issues. In comparison, syntax (B) and code generation (C) errors are significantly smaller, while runtime (D) and other (E) errors are negligible. This indicates the effectiveness of *TranLibEval* in the evaluation of TPL-involved code translation, exposing various kinds of failures that LLMs induce in the face of TPLs.

Figure 5 illustrates a horizontal comparison of error analysis among different translation strategies, where the Third-Party Reference (TPR) errors consistently account for the largest proportion. In terms of overall error volume, the *Direct* strategy and the *IR(CoT)* strategy yielded significantly

Table 6. Error categories vs. translation directions and strategies

Categories	A	A-1	A-1-1	A-1-2	A-1-3	A-1-4	A-2	A-2-1	A-2-2	A-2-3	A-2-4	A-3	A-3-1	A-3-2	A-3-3	B	B-1	B-2	B-3	C	C-1	C-2	C-3	D	E
Direct																									
C++ → Java	73.31	19.26	6.76	1.35	8.78	2.36	32.77	10.81	0.68	17.91	3.38	**21.28**	9.12	5.07	7.09	3.04	1.69	1.35	0.0	14.19	0.34	10.14	3.72	6.76	2.7
C++ → Python	**84.14**	28.97	11.03	2.76	12.41	2.76	40.69	14.48	2.76	18.62	4.83	14.48	6.21	4.14	4.14	1.38	1.38	0.0	0.0	10.34	0.0	8.28	2.07	4.14	0.0
Java → C++	72.57	27.88	10.84	2.21	10.84	3.98	29.87	12.39	0.22	15.04	2.21	14.82	6.42	3.32	5.09	3.54	0.88	2.21	0.44	**14.6**	0.0	13.72	0.88	8.19	1.11
Java → Python	69.7	24.24	7.58	1.52	11.36	3.79	**45.45**	12.88	0.76	31.82	0.0	0.0	0.0	0.0	0.0	1.52	1.52	0.0	0.0	12.12	0.0	6.82	5.3	13.64	3.03
Python → C++	80.22	34.26	16.16	3.34	10.86	3.9	28.97	12.26	2.79	12.26	1.67	16.99	6.96	5.01	5.01	4.18	1.95	1.67	0.56	5.57	0.0	3.62	1.95	7.52	2.51
Python → Java	74.74	**41.3**	19.11	4.1	11.95	6.14	22.53	12.97	2.39	6.14	1.02	10.92	4.78	3.41	2.73	**20.82**	1.02	1.02	18.77	0.34	0.34	0.0	0.0	3.75	0.34
RA (method)																									
C++ → Java	70.3	30.3	11.52	2.42	11.52	4.85	35.15	13.94	3.64	16.36	1.21	4.85	0.61	0.61	3.64	1.21	1.21	0.0	0.0	15.15	0.0	15.15	0.0	12.73	0.61
C++ → Python	66.46	20.5	6.83	1.86	8.07	3.73	38.51	9.94	6.21	20.5	1.86	7.45	2.48	2.48	2.48	1.24	1.24	0.0	0.0	19.25	0.0	14.91	4.35	11.8	1.24
Java → C++	68.94	**32.95**	17.05	3.41	9.47	3.03	22.35	10.98	3.79	0.0	7.58	13.64	4.55	3.41	5.68	**18.56**	1.14	12.5	4.92	12.5	0.0	0.38	12.12	0.0	0.0
Java → Python	73.44	17.19	4.69	1.56	8.59	2.34	34.38	9.38	7.03	17.97	0.0	**21.88**	11.72	1.56	8.59	1.56	1.56	0.0	0.0	**21.09**	0.78	17.19	3.12	2.34	1.56
Python → C++	62.96	23.28	10.58	2.12	6.88	3.7	27.51	10.05	2.65	6.35	8.47	12.17	5.82	2.65	3.7	15.34	0.53	12.17	2.65	19.58	0.0	1.06	18.52	2.12	0.0
Python → Java	**77.14**	27.43	9.14	2.29	13.71	2.29	**45.71**	15.43	1.71	25.14	3.43	4.0	0.57	2.86	0.57	3.43	0.57	2.29	0.57	18.29	0.0	5.71	12.57	0.57	0.57
RA (name)																									
C++ → Java	84.38	28.82	10.07	2.08	13.89	2.78	**50.69**	14.93	0.69	34.38	0.69	4.86	1.74	1.74	1.39	3.12	1.74	0.35	1.04	7.64	0.0	4.86	2.78	4.51	0.35
C++ → Python	**89.89**	24.47	7.45	1.6	13.3	2.13	48.94	14.89	0.53	30.85	2.66	16.49	6.91	3.72	5.85	0.0	0.0	0.0	0.0	8.51	0.0	3.19	5.32	1.06	0.53
Java → C++	87.39	34.78	13.48	3.04	13.04	5.22	34.35	13.91	2.61	16.52	1.3	**18.26**	8.7	3.48	6.09	**6.52**	1.74	0.0	4.78	3.48	0.0	1.3	2.17	1.74	0.87
Java → Python	75.3	16.27	3.01	0.6	11.45	1.2	48.8	11.45	1.2	36.14	0.0	10.24	6.02	1.2	3.01	3.61	3.61	0.0	0.0	**20.48**	0.0	19.28	1.2	0.0	0.6
Python → C++	86.9	**36.24**	13.97	3.06	13.97	5.24	38.43	14.85	0.87	21.4	1.31	12.23	4.8	3.06	4.37	5.68	2.18	0.0	3.49	6.11	0.0	2.62	3.49	0.87	0.44
Python → Java	86.72	29.46	10.79	2.49	13.28	2.9	50.21	14.11	2.9	32.78	0.41	7.05	2.9	1.66	2.49	2.9	0.83	0.0	2.07	6.64	0.0	4.15	2.49	3.32	0.41
IR (CoT)																									
C++ → Java	89.28	31.37	12.33	2.68	13.67	2.68	39.14	14.21	0.8	24.13	0.0	18.77	5.9	5.9	6.97	3.49	2.14	0.27	1.07	4.02	1.61	0.0	2.41	2.95	0.27
C++ → Python	76.34	16.13	3.23	1.08	10.75	1.08	48.92	10.75	3.76	34.41	0.0	11.29	3.76	3.76	3.76	0.54	0.0	0.0	0.54	**22.58**	0.0	3.23	19.35	0.0	0.54
Java → C++	61.11	18.22	8.0	1.78	6.44	2.0	18.67	7.33	2.0	7.11	2.22	24.22	9.78	6.22	8.22	**28.89**	12.67	1.78	14.44	8.67	0.0	0.0	8.67	0.44	0.89
Java → Python	75.52	9.79	1.4	0.7	6.29	1.4	33.57	8.39	4.2	16.78	4.2	**32.17**	14.69	6.29	11.19	0.0	0.0	0.0	0.0	20.98	0.0	0.0	20.98	3.5	0.0
Python → C++	68.86	29.71	13.14	2.86	11.14	2.57	25.43	11.71	0.57	11.71	1.43	13.71	4.86	4.29	4.57	23.43	11.71	2.29	9.43	6.29	0.0	0.0	6.29	0.86	0.57
Python → Java	**94.51**	**35.71**	12.09	2.75	17.03	3.85	**51.1**	18.13	0.55	31.87	0.55	7.69	2.75	2.2	2.75	2.75	0.0	0.0	2.75	1.65	0.0	1.65	0.0	0.55	0.55
IR (pseudocode)																									
C++ → Java	83.24	21.79	5.59	1.12	13.41	1.68	**54.75**	15.08	0.0	36.87	2.79	6.7	1.68	1.68	3.35	2.79	0.0	0.0	2.79	13.41	0.0	7.82	5.59	0.56	0.0
C++ → Python	70.83	21.53	8.33	2.08	9.72	1.39	34.03	9.72	4.17	20.14	0.0	15.28	5.56	5.56	4.17	0.0	0.0	0.0	0.0	**29.17**	0.0	7.64	21.53	0.0	0.0
Java → C++	85.06	38.31	15.58	3.25	14.61	4.87	36.69	15.26	0.32	20.45	0.65	10.06	3.57	3.57	2.92	5.52	0.65	1.3	3.57	6.49	0.32	3.25	2.92	1.62	1.3
Java → Python	72.03	24.58	9.32	2.12	9.75	3.39	24.58	10.17	2.97	11.02	0.42	**22.88**	11.02	3.81	8.05	**13.56**	2.12	11.44	0.0	13.98	0.0	9.75	4.24	0.0	0.42
Python → C++	**90.44**	**39.34**	19.85	4.04	11.76	3.68	31.25	12.5	1.1	15.44	2.21	19.85	9.19	3.68	6.99	1.47	0.37	0.37	0.74	5.88	0.37	4.41	1.1	1.1	1.1
Python → Java	89.18	31.6	12.12	2.6	13.85	3.03	48.92	14.72	0.43	32.9	0.87	8.66	3.03	2.16	3.46	1.73	0.87	0.43	0.43	8.23	1.73	5.63	0.87	0.43	0.43
IR (summary)																									
C++ → Java	**98.44**	24.92	6.23	1.25	15.89	1.56	**68.22**	15.89	3.12	49.22	0.0	5.3	2.49	1.25	1.56	1.25	0.0	0.31	0.93	0.31	0.0	0.31	0.0	0.0	0.0
C++ → Python	95.98	**36.61**	12.05	2.68	18.3	3.57	54.91	18.75	2.68	33.48	0.0	4.46	1.79	1.34	1.34	0.45	0.0	0.45	0.0	1.79	0.0	1.79	0.0	1.34	0.45
Java → C++	77.65	28.24	12.94	2.65	10.0	2.65	36.47	12.35	1.18	15.59	7.35	**12.94**	5.59	2.65	4.71	10.29	7.94	0.29	2.06	11.47	0.0	1.47	10.0	0.29	0.29
Java → Python	87.18	17.52	2.56	0.85	13.25	0.85	60.68	13.68	4.27	42.74	0.0	8.97	3.42	2.56	2.99	0.0	0.0	0.0	0.0	**11.54**	0.0	0.43	11.11	1.28	0.0
Python → C++	71.97	26.23	10.54	2.24	10.76	2.69	33.18	11.66	0.45	18.61	2.47	12.56	4.71	3.59	4.26	**15.47**	10.09	5.38	0.0	8.74	0.22	0.0	8.52	2.47	1.35
Python → Java	88.54	31.85	12.74	2.55	13.38	3.18	54.78	15.92	4.46	24.2	10.19	1.91	0.64	0.64	0.64	8.28	3.18	2.55	2.55	3.18	1.91	1.27	0.0	0.0	0.0

higher counts, ranging from 1,600 to over 1,700 cases. In contrast, retrieval-augmented methods remained in a lower range, between 1,000 and 1,400 errors.

In the *Direct* strategy, TPR errors account for 74.77% of all errors, with a significant portion arising from library-related errors, indicating that the *Direct* strategy fails in the library importation stage, not to mention calling APIs, which leads to the top tier in terms of the number of errors. The *IR(summary)* strategy shows the highest rate of API-related errors (A-2), particularly the use of calling an irrelevant API (A-2-3), accounting for 30.63% of its TPR errors, suggesting that while summarization aids in capturing high-level logic, it still struggles with accurate API alignment across programming environments. The *IR(CoT)* strategy exhibits a higher proportion of parameter-related errors (A-3), especially parameter type mismatches (A-3-1) and return value errors (A-3-3), comprising 21.00% of its TPR issues. The incorporation of chain-of-thought reasoning appears to introduce additional complexity in parameter mapping, despite its detailed reasoning process.

Among all strategies, retrieval-augmented strategies, especially *RA(method)*, produce the fewest errors, benefiting from broader contextual retrieval for better API and library matching. Although *IR(pseudocode)* yields the best results among IR strategies, it only performs on par with *RA(name)*. The former often loses critical implementation details during abstraction, while the latter is limited by retrieval inaccuracies. These observations underscore the persistent difficulty of aligning third-party references in cross-lingual code translation and highlight the specific improvement directions and precautions for use of the above translation strategies.

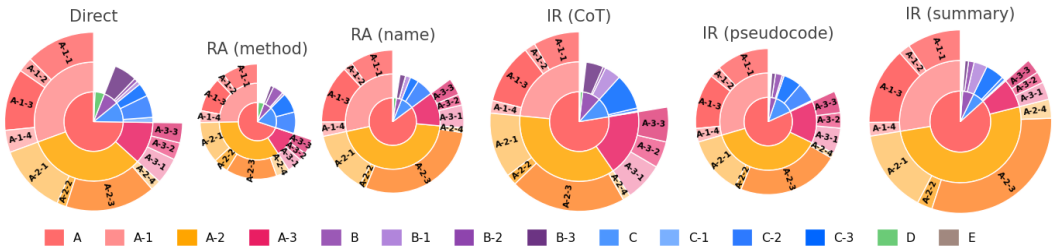


Fig. 5. A Horizontal Comparison of Error Analysis among Different Translation Strategies

Finding 9: Although third-party reference errors are prevalent across all translation strategies, their distributions reveal distinct patterns: the *Direct* strategy suffers most from library errors (A-1), *IR(summary)* peaks in API errors (A-2), and *IR(CoT)* exhibits high parameter errors (A-3). Retrieval-augmented methods, particularly *RA(method)*, achieve the lowest error rates. While *IR(pseudocode)* performs best among IR variants, it only matches *RA(name)*, with limitations in abstraction and retrieval scope, respectively. These findings underscore the need for more robust handling of external library references and precautions for use of the above translation strategies.

6 Implications

Implications for researchers: (1) As the inaugural study to concentrate on TPL utilization in code translation, this work brings to light fundamental limitations in existing benchmarks, which typically lack high categorical coverage and large-scale TPL-targeted code samples. (2) According to the well-known failures involving TPLs/APIs [40, 67, 73], we introduce five translation strategies under two widely applicable and highly motivated methodological systems, i.e., IR-guided and retrieval-augmented, and make comparisons with the *Direct* strategy in RQ2. Among the five strategies, *IR(pseudocode)*, *RA(method)*, and *RA(name)* are the first time to be applied in the code translation field for addressing TPL-related failures, offering basic design principles and usage experience for this area's future research. (3) Afterwards, we explain the results of RQ1-2 with multiple views (LLMs, translation directions, and strategies) by delving into the library dependency awareness, significantly improving academia's understanding of why certain LLMs perform better with certain strategies on certain translation directions. (4) Furthermore, RQ4 identifies a spectrum of failure patterns covering 19 categories, where 11 of them are unique to library-centric scenarios, such as calling an invalid library (A-1-1) and calling an irrelevant API (A-2-3). These issues are frequently obscured in prior studies [24, 35, 62, 64, 67, 73] due to their benchmarks' limitations mentioned above in (1). Thus, our work provides more explicit directions for future code translation research.

Implications for practitioners: (1) We constructed the first TPL-targeted code translation benchmark, *TransLibEval*, which consists of 200 real-world TPL-involved samples for industrial evaluation. (2) Through *TransLibEval*, we conducted extensive experiments to investigate the performance of diverse LLMs with different translation strategies on various translation directions among mainstream PLs, i.e., Python, Java, and C++, in RQ1-3, revealing the heterogeneous advantages of diverse LLMs and translation strategies as well as offering suggestions and explanations for developers' practical usage. (3) Moreover, 19 error types are manually categorized from 4,831 failures of GPT-4o, precisely reminding practitioners to notice its potential risks when deploying for TPL-intensive scenarios. Besides, the follow-up strategy-wise analysis further offers insightful suggestions from the perspectives of their frequent errors, helping practitioners to understand each strategy's limitations beyond their advantages in RQ2.

7 Threats to Validity

Threats to external validity relate to the generality of the findings and the failure taxonomy [32, 68]. Our evaluation is limited to code translation between Python, Java, and C++, excluding other PLs. However, *TransLibEval* is composed of larger-scale, richer-category coverage, practical TPL-centric tasks than previous work (e.g., [24, 29, 67]), making our results more applicable to real-world development. The dominance of Python, Java, and C++ in industry and academia [59] also confirms the practical implications of our findings. To construct the failure taxonomy, we analyzed a corpus of 4,831 distinct error cases from one of the SOTA LLMs. This extensive analysis, spanning all three PLs, six strategies, and six translation directions, guided by thematic analysis, substantially minimizes threats to the generality of our conclusions.

Threats to internal validity mainly involve potential data leakage problems and manual handling procedures of this work [24, 68]. To mitigate the data leakage problem, we manually constructed *TranLibEval*, which significantly reduces the possibility of being seen by LLMs. Moreover, a recent study [72] proved with extensive experiments that code translation tasks are almost unaffected by data leakage owing to their unpaired features of source-target codes in LLMs' pre-training. *TransLibEval* is intended to incorporate a self-refreshing design that regularly updates via automated program mutation, thereby maintaining its relevance, rigor, and data leakage-free over time. For tasks involving manual construction and inspection, multiple expert annotators performed a double-blind review following thematic analysis principles to minimize subjectivity and error. Hence, the above threats are minimal.

Threats to construct validity concern the comprehensiveness of evaluation metrics [13, 47]. To achieve this, we exert PR to evaluate the test case pass rate and CA to assess the pass rate of the whole test suite. Besides, we also examine their CSR from a compilation perspective. Moreover, we leverage LDA to investigate the ability of LLMs to correctly identify useful TPLs when translating code. We do not include literal consistency, e.g., exact match accuracy, for assessment because code translation involving third-party dependencies is way more challenging, and no sample can be translated exactly the same with ground truths, making such evaluation meaningless. In addition, semantically equivalent programs are not necessarily consistent in literal expressions. Hence, considering the above diverse evaluation workarounds, this threat is limited.

8 Conclusion and Future Work

This work constructs the first benchmark for evaluating TPL-targeted code translation, namely *TransLibEval*, and introduces 6 highly motivated translation strategies for assessment. Extensive experiments on 7 recent LLMs of diverse categories and sizes demonstrate their weaknesses on TPL-involved code translation and uncover the heterogeneous advantages of each strategy, offering valuable guidance for practical usage. Finally, we proceeded with a thorough manual analysis and categorization of LLMs' translation failures on third-party dependency samples, as the first attempt, shedding light for researchers and practitioners to facilitate their studies and applications.

Based on our findings, our future work aims to enhance the performance of LLMs in TPL-targeted code translation by further optimizing retrieval or IR-guided strategies. Additionally, we will extend *TransLibEval* to a broader range of PL pairs with strong industrial relevance, such as C++ to Rust.

Acknowledgments

This work was partially supported by the National Natural Science Foundation of China (Grant Nos. 62502283 and U24B20149), the Natural Science Foundation of Shandong Province (Grant No. ZR2024QF093), and the Young Talent of Lifting Engineering for Science and Technology in Shandong, China (Grant No. SDAST2025QTB031).

Data Availability: We open-sourced the replication package and *TransLibEval* benchmark at [3].

References

[1] [n. d.]. Codeforces. https://codeforces.com/ [Online; accessed 2025-02-21].

[2] [n. d.]. Stack Exchange API. https://api.stackexchange.com/docs [Online; accessed 2025-09-04].

[3] [n. d.]. Xiaozheng03Asada/TransLibEval. https://github.com/Xiaozheng03Asada/TransLibEval [Online; accessed 2026-01-17].

[4] 2025. OpenAI. https://openai.com/ [Online; accessed 2025-02-17].

[5] 2026. *Error Analysis Report.* Technical Report. GitHub Repository. https://github.com/Xiaozheng03Asada/TransLibEval/blob/main/Failed_Cases_Report.pdf [Online; accessed 2026-01-17].

[6] Asma Ben Abacha, Wen-wai Yim, Yujuan Fu, Zhaoyi Sun, Meliha Yetisgen, Fei Xia, and Thomas Lin. 2024. Medec: A benchmark for medical error detection and correction in clinical notes. *arXiv preprint arXiv:2412.19260* (2024).

[7] Wasi Ahmad, Md Golam Rahman Tushar, Saikat Chakraborty, and Kai-Wei Chang. 2023. AVATAR: A Parallel Corpus for Java-Python Program Translation. In *Findings of the Association for Computational Linguistics: ACL 2023*. 2268–2281.

[8] Wasi Uddin Ahmad, Svyatoslav Pestov, M Saiful Bari, Zadia Codabux, Imad Eddine Marouf, Rohan Saphal, Raphael Rubino, and Bonan Min. 2022. AVATAR: A Parallel Corpus for Code Generation with Variable-Level Granularity. In *Findings of the Association for Computational Linguistics: EMNLP 2022*. Association for Computational Linguistics, 495–509. doi:10.18653/v1/2022.findings-emnlp.34

[9] Reem Alsuhaibani, Christian Newman, Michael Decker, Michael Collard, and Jonathan Maletic. 2021. On the naming of methods: A survey of professional developers. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 587–599.

[10] Apache Maven. 2024. Maven - Java Project Management and Comprehension Tool. https://maven.apache.org/ (Accessed on 10/31/2024).

[11] Baidu Qianfan. 2024. Baidu Qianfan Platform - Open API Interface. https://qianfan.cloud.baidu.com/ (Accessed on 10/31/2024).

[12] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alexey Strokach, Wojciech Zaremba, John Schulman, and Ilya Sutskever. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374 [cs.LG] doi:10.48550/ARXIV.2107.03374

[13] Zachary Coker, Zixuan Li, Shiqi Wang, Yuchen Morgann Chen, Jie M. Zhang, Anderson R. Avila, David H. Smith, and Ahmed E. Hassan. 2023. A Methodology for Building Trustworthy Code Generation Benchmarks. In *Proceedings of the 45th International Conference on Software Engineering (ICSE)*. IEEE, 2124–2135. doi:10.1109/ICSE48619.2023.00185

[14] Deepseek. 2024. Deepseek - Official API Interface. https://www.deepseek.com/zh (Accessed on 10/31/2024).

[15] DeepSeek Team. 2024. DeepSeek. https://www.deepseek.com/. (Accessed on 10/27/2024).

[16] Gabriel Dos Reis and Bjarne Stroustrup. 2006. Specifying C++ concepts. *ACM SIGPLAN Notices* 41, 1 (2006), 295–308.

[17] Xueying Du, Mingwei Liu, Kaixin Wang, Hanlin Wang, Junwei Liu, Yixuan Chen, Jiayi Feng, Chaofeng Sha, Xin Peng, and Yiling Lou. 2024. Evaluating large language models in class-level code generation. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13.

[18] Ozkan Kemal Eniser, Zih-Syuan Huang, An-Da Iacob, Chun-Fu Chen, and Pin-Yu Chen. 2023. Is Rust the Right Language for LLM-Based Code Translation?. In *Proceedings of the 45th International Conference on Software Engineering (ICSE)*. IEEE, 1967–1979. doi:10.1109/ICSE48619.2023.00171

[19] Christopher Flynn. 2024. PyPI Download Stats. *URL: https://pypistats. org/top* (2024).

[20] Yi Gao, Xing Hu, Tongtong Xu, Xin Xia, and Xiaohu Yang. 2024. A Rule-Based Approach for UI Migration from Android to iOS. *arXiv preprint arXiv:2409.16656* (2024).

[21] GitHub Octoverse. 2024. The top programming languages | The State of the Octoverse. https://octoverse.github.com/2022/top-programming-languages?src_trk=em6690b43a717440.49181083182294122 6. (Accessed on 10/27/2024).

[22] Google. 2024. GoogleTest - C++ Testing Framework. https://google.github.io/googletest/ (Accessed on 10/31/2024).

[23] Kaifeng Huang, Bihuan Chen, Congying Xu, Ying Wang, Bowen Shi, Xin Peng, Yijian Wu, and Yang Liu. 2022. Characterizing usages, updates and risks of third-party libraries in Java projects. *Empirical Software Engineering* 27, 4 (2022), 90.

[24] Ali Reza Ibrahimzada, Kaiyao Ke, Mrigank Pawagi, Muhammad Salman Abid, Rangeet Pan, Saurabh Sinha, and Reyhaneh Jabbarvand. 2025. AlphaTrans: A Neuro-Symbolic Compositional Approach for Repository-Level Code Translation and Validation. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 2454–2476.

[25] Xue Jiang, Yihong Dong, Lecheng Wang, Zheng Fang, Qiwei Shang, Ge Li, Zhi Jin, and Wenpin Jiao. 2024. Self-planning code generation with large language models. *ACM Transactions on Software Engineering and Methodology* 33, 7 (2024), 1–30.

[26] Junda Jiao, Yelin Liu, Zhuo Liu, Yuxin Wang, Yuxin Lu, Yuchen Yao, Binyuan Hui, Ge Li, and Zhi Jin. 2024. G-TransEval: A Granular-grained Code Translation Benchmark for General-purpose Languages. arXiv:2403.01831 [cs.SE] doi:10.48550/ARXIV.2403.01831

[27] Mingsheng Jiao, Tingrui Yu, Xuan Li, Guanjie Qiu, Xiaodong Gu, and Beijun Shen. 2023. On the evaluation of neural code translation: Taxonomy and benchmark. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 1529–1541.

[28] JUnit 5. 2024. JUnit 5 - Testing Framework for Java. https://junit.org/junit5/ (Accessed on 10/31/2024).

[29] Mohammad Abdullah Matin Khan, M Saiful Bari, Do Long, Weishi Wang, Md Rizwan Parvez, and Shafiq Joty. 2024. Xcodeeval: An execution-based large scale multilingual multitask benchmark for code understanding, generation, translation and retrieval. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 6766–6805.

[30] Jia Li, Ge Li, Yongmin Li, and Zhi Jin. 2024. Structured Chain-of-Thought Prompting for Code Generation. *ACM Trans. Softw. Eng. Methodol.* (Aug. 2024). doi:10.1145/3690635 Just Accepted.

[31] Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Dengr, Chong Ruan, Damai Dai, Daya Guo, et al. 2024. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434* (2024).

[32] Fang Liu, Yang Liu, Lin Shi, Houkun Huang, Ruifeng Wang, Zhen Yang, and Li Zhang. 2024. Exploring and evaluating hallucinations in llm-powered code generation. *arXiv preprint arXiv:2404.00971* (2024).

[33] Llama Team. 2024. Llama 3. https://www.llama.com/. (Accessed on 10/27/2024).

[34] Sheng Lu, An-Da Iacob, Chun-Fu Chen, Pin-Yu Chen, Jin-Hee-Lee, Gevorgyan Vahagn, Yuxin Wang, Yufan Lu, Bolin Wei, Cheng-Hao Tsai, Yijun Wang, Kelan Jiang, Haidong Zhang, Yuchi Tian, Han-Pang Chiu, Ge Li, and Yun-Nung Chen. 2023. xCodeEval: A Large-Scale Multilingual Benchmark for Code Generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, 14107–14125. doi:10.18653/v1/2023.emnlp-main.873

[35] Feng Luo, Kexing Ji, Cuiyun Gao, Shuzheng Gao, Jia Feng, Kui Liu, Xin Xia, and Michael R Lyu. 2025. Integrating Rules and Semantics for LLM-Based C-to-Rust Translation. *arXiv preprint arXiv:2508.06926* (2025).

[36] NuGet. 2024. NuGet - Package Manager for .NET. https://www.nuget.org/ (Accessed on 10/31/2024).

[37] Guangsheng Ou, Mingwei Liu, Yuxuan Chen, Xing Peng, and Zibin Zheng. 2024. Repository-level Code Translation Benchmark Targeting Rust. *arXiv preprint arXiv:2411.13990* (2024).

[38] Guangsheng Ou, Mingwei Liu, Yuxuan Chen, Xing Peng, and Zibin Zheng. 2025. Enhancing LLM-based Code Translation in Repository Context via Triple Knowledge-Augmented. *arXiv preprint arXiv:2503.18305* (2025).

[39] Jialing Pan, Adrien Sadé, Jin Kim, Eric Soriano, Guillem Sole, and Sylvain Flamant. 2023. SteloCoder: a Decoder-Only LLM for Multi-Language to Python Code Translation. *arXiv preprint arXiv:2310.15539* (2023).

[40] Rangeet Pan, Ali Reza Ibrahimzada, Rahul Krishna, Divya Sankar, Lambert Pouguem Wassi, Michele Merler, Boris Sobolev, Raju Pavuluri, Saurabh Sinha, and Reyhaneh Jabbarvand. 2024. Lost in translation: A study of bugs introduced by large language models while translating code. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13.

[41] Ting Pan, Zhaohua Yan, Zirui Chen, Peilin Zhao, Yanchi Li, Haifeng Sun, Jian-Guang Lou, and Wei Zhang. 2023. On the Effectiveness of Large Language Models for Code Translation. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. ACM, 110–121. doi:10.1145/3611643.3616239

[42] PEP8. [n. d.]. PEP 8 – Style Guide for Python Code | peps.python.org. https://peps.python.org/pep-0008/#descriptive-naming-styles [Online; accessed 2025-09-01].

[43] Lutz Prechelt. 2000. An empirical comparison of c, c++, java, perl, python, rexx and tcl. *IEEE Computer* 33, 10 (2000), 23–29.

[44] Ruchir Puri, David S. Kung, Geert S. T. Brocato, et al. 2021. CodeNet: A Large-Scale AI for Code Dataset for Learning a Diversity of Coding Tasks. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*. https://proceedings.neurips.cc/paper/2021/hash/1457c0d6bfcb496d518339378c2d7653-Abstract.html

[45] Python Documentation. 2024. unittest - Python's Unit Testing Framework. https://docs.python.org/3/library/unittest.html (Accessed on 10/31/2024).

[46] Achut Reddy et al. 2000. Java™ coding style guide. *Sun MicroSystems* (2000).

[47] Debarshi Roy and Marius Litoiu. 2023. A Survey on Evaluation Metrics for Large Language Models of Code. arXiv:2305.07439 [cs.SE] doi:10.48550/ARXIV.2305.07439

[48] Baptiste Rozière, Marie-Anne Lachaux, Lowik Chanussot, and Guillaume Lample. 2020. Unsupervised translation of programming languages. In *Proceedings of the 34th Conference on Neural Information Processing Systems (NeurIPS)*. 15666–15676. https://proceedings.neurips.cc/paper/2020/hash/99A4104C333A15F16913f00593498A44-Abstract.html

[49] Baptiste Roziere, Marie-Anne Lachaux, Lowik Chanussot, and Guillaume Lample. 2020. Unsupervised translation of programming languages. *Advances in Neural Information Processing Systems* 33 (2020), 20601–20611. doi:10.48550/arXiv.2006.03511

[50] Baptiste Roziere, Jie Zhang, Francois Charton, Mark Harman, Gabriel Synnaeve, and Guillaume Lample. 2021. Leveraging Automated Unit Tests for Unsupervised Code Translation. In *International Conference on Learning Representations*. doi:10.48550/arXiv.2110.06773

[51] Roziere, Baptiste and Gehring, Jonas and Gloeckle, Fabian and Sootla, Sten and Gat, Itai and Tan, Xiaoqing Ellen and Adi, Yossi and Liu, Jingyu and Sauvestre, Romain and Remez, Tal and others. 2023. Code Llama: Open Foundation Models for Code. *arXiv preprint arXiv:2308.12950* (2023).

[52] Stack Overflow. 2024. Stack Overflow - Programming Q&A Site. https://stackoverflow.com/ (Accessed on 10/31/2024).

[53] Herb Sutter and Andrei Alexandrescu. 2004. *C++ coding standards: 101 rules, guidelines, and best practices*. Pearson Education.

[54] Qingxiao Tao, Tingrui Yu, Xiaodong Gu, and Beijun Shen. 2024. Unraveling the Potential of Large Language Models in Code Translation: How Far Are We? *arXiv preprint arXiv:2410.09812* (2024).

[55] Zhiruo Tao, Yida Zhai, Zifeng Wang, Kailong Wang, Hong-Ning Dai, Yong Li, and Zhenhong Chen. 2023. Polyglot Human-Level Programming with Large Language Models. arXiv:2311.08225 [cs.CL] doi:10.48550/ARXIV.2311.08225

[56] CodeGemma Team. 2024. Codegemma: Open code models based on gemma. *arXiv preprint arXiv:2406.11409* (2024).

[57] Gemma Team, Thomas Mesnard, Cassidy Hardin, Robert Dadashi, Surya Bhupatiraju, Shreya Pathak, Laurent Sifre, Morgane Rivière, Mihir Sanjay Kale, Juliette Love, et al. 2024. Gemma: Open models based on gemini research and technology. *arXiv preprint arXiv:2403.08295* (2024).

[58] Qwen Team. 2025. Qwen. https://qwen.ai/home. https://qwen.ai/home Accessed: 2025-08-28.

[59] TIOBE Software. 2025. TIOBE Programming Community Index. https://www.tiobe.com/tiobe-index/. Accessed: July 2025.

[60] Frank Tip, Jonathan Bell, and Max Schäfer. 2025. Llmorpheus: Mutation Testing Using Large Language Models. *IEEE Transactions on Software Engineering* (2025).

[61] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288* (2023).

[62] Chaofan Wang, Guanjie Qiu, Xiaodong Gu, and Beijun Shen. 2025. APIRAT: Integrating Multi-source API Knowledge for Enhanced Code Translation with LLMs. *arXiv preprint arXiv:2504.14852* (2025).

[63] Chaofan Wang, Tingrui Yu, Jie Wang, Dong Chen, Wenrui Zhang, Yuling Shi, Xiaodong Gu, and Beijun Shen. 2025. EVOC2RUST: A Skeleton-guided Framework for Project-Level C-to-Rust Translation. *arXiv preprint arXiv:2508.04295* (2025).

[64] Yanli Wang, Yanlin Wang, Suiquan Wang, Daya Guo, Jiachi Chen, John Grundy, Xilin Liu, Yuchi Ma, Mingzhi Mao, Hongyu Zhang, et al. 2024. RepoTransBench: A Real-World Benchmark for Repository-Level Code Translation. *arXiv preprint arXiv:2412.17744* (2024).

[65] Peter Wentworth, Jeffrey Elkner, Allen B Downey, and Chris Meyers. 2011. How to think like a computer scientist. (2011).

[66] Li Xin-Ye, Du Ya-Li, and Li Ming. 2025. Enhancing LLMs in Long Code Translation through Instrumentation and Program State Alignment. *arXiv preprint arXiv:2504.02017* (2025).

[67] Pengyu Xue, Linhao Wu, Zhen Yang, Chengyi Wang, Xiang Li, Yuxiang Zhang, Jia Li, Ruikai Jin, Yifei Pei, Zhaoyan Shen, et al. 2025. ClassEval-T: Evaluating Large Language Models in Class-Level Code Translation. *Proceedings of the ACM on Software Engineering* 2, ISSTA (2025), 1421–1444.

[68] Pengyu Xue, Linhao Wu, Zhongxing Yu, Zhi Jin, Zhen Yang, Xinyi Li, Zhenyu Yang, and Yue Tan. 2024. Automated commit message generation with large language models: An empirical study and beyond. *IEEE Transactions on Software Engineering* (2024).

[69] Fei Yan, Zhisong Olden, Hongyu Zhang, Tianyi Zhou, and Ge Li. 2024. CodeTransOcean: A Large-Scale Benchmark for Code Translation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*. Association for Computational Linguistics, 837–850. https://aclanthology.org/2024.eacl-long.50

[70] Weixiang Yan, Yuchen Tian, Yunzhe Li, Qian Chen, and Wen Wang. 2023. Codetransocean: A comprehensive multilingual benchmark for code translation. *arXiv preprint arXiv:2310.04951* (2023).

[71] Zhen Yang, Jacky Keung, Xiao Yu, Xiaodong Gu, Zhengyuan Wei, Xiaoxue Ma, and Miao Zhang. 2021. A multi-modal transformer-based code summarization approach for smart contracts. In *2021 IEEE/ACM 29th International Conference on Program Comprehension (ICPC)*. IEEE, 1–12.

[72] Zhen Yang, Hongyi Lin, Yifan He, Jie Xu, Zeyu Sun, Shuo Liu, Pengpeng Wang, Zhongxing Yu, and Qingyuan Liang. 2025. Rethinking the effects of data contamination in Code Intelligence. *arXiv preprint arXiv:2506.02791* (2025).

[73] Zhen Yang, Fang Liu, Zhongxing Yu, Jacky Wai Keung, Jia Li, Shuo Liu, Yifan Hong, Xiaoxue Ma, Zhi Jin, and Ge Li. 2024. Exploring and unleashing the power of large language models in automated code translation. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 1585–1608.

[74] Xin Yin, Chao Ni, Tien N Nguyen, Shaohua Wang, and Xiaohu Yang. 2024. Rectifier: Code translation with corrector via llms. *arXiv preprint arXiv:2407.07472* (2024).

[75] Hao Yu, Bo Shen, Dezhi Ran, Jiaxin Zhang, Qi Zhang, Yuchi Ma, Guangtai Liang, Ying Li, Qianxiang Wang, and Tao Xie. 2024. Codereval: A benchmark of pragmatic code generation with generative pre-trained models. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–12.

[76] Puzhuo Yuan, Zhaohua Yan, Zirui Chen, Yanchi Li, Jian-Guang Lou, and Peilin Zhao. 2024. TRANSAGENT: A Multi-agent System for Code Translation. In *Proceedings of the 46th International Conference on Software Engineering (ICSE)*. ACM, 1640–1652. doi:10.1145/3639478.3639556

[77] Zhiqiang Yuan, Weitong Chen, Hanlin Wang, Kai Yu, Xin Peng, and Yiling Lou. 2024. Transagent: An llm-based multi-agent system for code translation. *arXiv preprint arXiv:2409.19894* (2024).

[78] Zhiqiang Yuan, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, Xin Peng, and Yiling Lou. 2024. Evaluating and improving chatgpt for unit test generation. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 1703–1726.

[79] Yang Zhao, Li Du, Xiao Ding, Kai Xiong, Zhouhao Sun, Jun Shi, Ting Liu, and Bing Qin. 2024. Deciphering the Impact of Pretraining Data on Large Language Models through Machine Unlearning. *arXiv preprint arXiv:2402.11537* (2024).

[80] Bo Zhou, Jiaqi Shi, Ying Wang, Li Li, Tsz On Li, Hai Yu, and Zhiliang Zhu. 2025. Porting Software Libraries to OpenHarmony: Transitioning from TypeScript or JavaScript to ArkTS. *Proceedings of the ACM on Software Engineering* 2, ISSTA (2025), 1445–1466.

Received 2025-09-10; accepted 2025-12-22